Introduction

A fundamental problem of Software Engineering is the problem of scale. As systems grew more complex, the goals of the entire system could not be easily figure out.

The software project is initiated by the client's need. In the beginning the needs are in the minds of various people in the organization.

The requirement analyst has to identify the requirements by talking to these people and understand their needs.

The requirement phase translates the ideas in the mind of the clients into a formal document. The developers and clients basically interact in requirement analysis phase.

Software Requirements: (definition)

IEEE defines a requirement as

"(1) A condition of capability needed by a user to solve a problem or achieve an objective;

(2) A condition or a capability that must be met or possessed by a system ... to satisfy a contract, standard, specification, or other formally imposed document"

The goal of the requirements activity is to produce the Software Requirements Specification (SRS) that describes what the proposed software should do without describing how the software will do it.

Need for SRS

A basic purpose of software requirement specification is to bridge the communication gap between client and developer. Another important purpose of developing the SRS is helping the clients to understand their own needs. The process of developing SRS makes client & users to think, visualize, interact and discuss with others and to identify the requirements.

- *An SRS establishes the basis for agreement between the client and the developer on what the software will do.*

This basis for agreement is frequently formalized into legal contract between the client and the developer. So, through SRS, the client clearly describes what it expects from the supplier, and the developer clearly understands what capabilities to build in the software.

- *An SRS provides a reference for validation of the final product.*

SRS helps the client to determine if the software meets the requirements. Without a proper SRS, there is no way a client can determine if the software being delivered is what was ordered, and there is no way the developer can convince the client that all the requirements have been met.

- *A high-quality SRS is a prerequisite to high-quality software.*

The quality of SRS has an impact on cost (and schedule) of the project. The cost of fixing the errors increases as we proceed on to the further phases in the software development life cycle. In order to reduce the errors the SRS should be high quality.

- *A high-quality SRS reduces the development cost.*

If the SRS is of high quality, effort required to design the solution, effort required for coding, testing and maintenance will be reduced. Hence a high quality SRS reduces the development cost.

Requirement Process:

The requirement processes is the sequence of activities that need to be performed in the requirement phase. There are three basis activities in case of requirement analysis. They are-

1. Problem Analysis or Requirement Analysis.
2. Requirement Specification.
3. Requirement Validation.

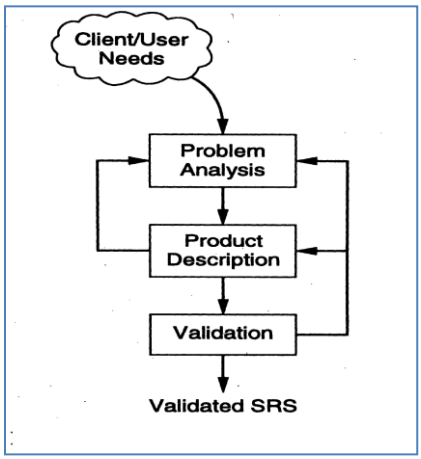


Fig: The requirement process

Problem analysis is initiated with some general statement of needs. Client is the originator of these needs. During analysis, the system behavior constrains on the system, its input outputs are analyzed.

The requirement specification clearly specifies the requirements in the form of document. The final activity focuses on validation of the collected requirements. Requirement process terminates with the production of validated SRS.

In most real systems, there is considerable overlap and feedback between these activities. So, some parts of the system are analyzed and then specified while the analysis of the other parts is going on. Furthermore, if the validation activities reveal problems in the SRS, it is likely to lead to further analysis and specification. However, in general, for a part of the system, analysis precedes specification and specification precedes validation.

During requirements analysis the focus is on understanding the system and its requirements. For a complex system, this is a hard task, and the time-tested method of "divide-and-conquer,"

i.e., decomposing the problem or system into smaller parts and then understanding the parts and their relationships.

The figure shows from the specification activity we may go back to the analysis activity. This happens as frequently some parts of the problem are analyzed and then specified before other parts are analyzed and specified.

Furthermore, the process of specification frequently shows shortcomings in the knowledge of the problem, thereby necessitating further analysis. Once the specification is "complete" it goes through the validation activity. This activity may reveal problems in the specifications itself, which requires going back to the specification step, or may reveal shortcomings in the understanding of the problem, which requires going back to the analysis activity.

Problem analysis:

The basic aim of problem analysis is to obtain a clear understanding of the needs of the clients and the users, what exactly is desired from the software, and what the constraints on the solution. The analysts have to ensure that the real needs of the clients and the users are uncovered, even if they don't know them clearly.

The basic principle used in analysis is the same as in any complex task: divide and conquer. That is, partition the problem into sub problems and then try to understand each sub problem and its relationship to other sub problems in an effort to understand the total problem.

The concepts of *state* and *projection* can sometimes also be used effectively in the partitioning process.

A state of a system represents some conditions about the system. In *projection,* a system is defined from multiple points of view.

Informal Approach

The informal approach to analysis is one where no defined methodology is used. Like in any approach, the information about the system is obtained by interaction with the client, end users, questionnaires, study of existing documents, brainstorming, etc. However, with this approach no formal model is built of the system. The problem and the system model are essentially built in the minds of the analysts and are directly translated from the minds of the analysts to the SRS.

Data Flow Modeling

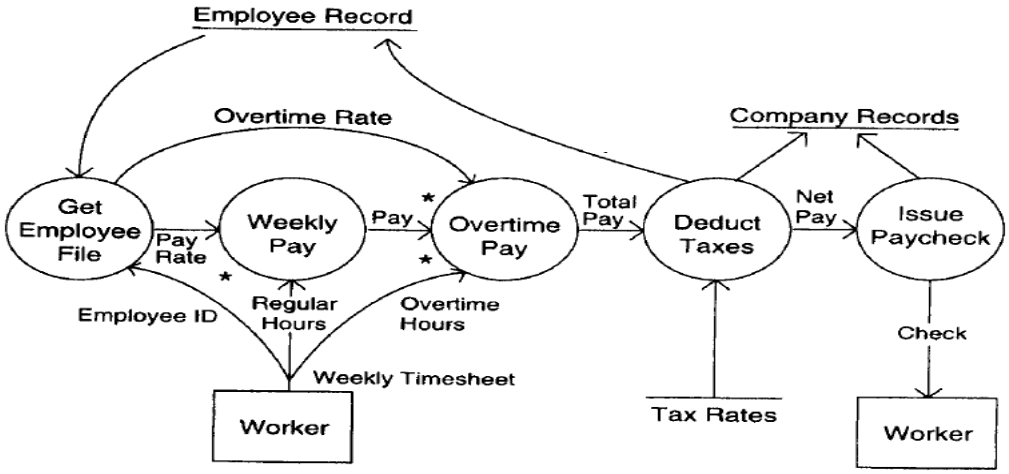
Data-flow based modeling, often referred to as the structured analysis function-based decomposition while modeling the problem. It focuses on the functions performed in the problem domain and the data consumed and produced by these functions. It is a top-down refinement approach, which was originally called structured analysis and specification, and was proposed for producing the specifications.

Data Flow Diagrams and Data Dictionary

Data flow diagrams (also called data flow graphs) are commonly used during problem analysis. Data flow diagrams (DFDs) are quite general and are not limited to problem analysis for software requirements specification.

DFDs are very useful in understanding a system and can be effectively used during analysis.
A DFD shows the flow of data through a system. It views a system as a function that transforms the inputs into desired outputs.
The DFD aims to capture the transformations that take place within a system to the input data so that eventually the output data is produced. The agent that performs the transformation of data from one state to another is called a process (or a bubble).
A DFD shows the movement of data through the different transformations or processes in the system. The processes are shown by named circles and data flows are represented by named arrows entering or leaving the bubbles. A rectangle represents a source or sinks and is a net originator or consumer of data. A source or a sink is typically outside the main system of study.

An example of a DFD for a system that pays workers is shown in Figure,



In this DFD there is one basic input data flow, the weekly timesheet, which originates from the source *worker*. The basic output is the paycheck, the sink for which is also the worker. In this system, first the employee's record is retrieved, using the employee ID, which is contained in the timesheet. From the employee record, the rate of payment and overtime are obtained.
After the total pay is determined, taxes are deducted. To compute the tax deduction, information from the
Tax-rate file is used. The amount of tax deducted is recorded in the employee and company records. Finally, the paycheck is issued for the net pay. The amount paid is also recorded in company records.

A DFD is not a flowchart. A DFD represents the flow of data, while a flowchart shows the flow of control. A DFD does not represent procedural information.
There are no detailed procedures that can be used to draw a DFD for a given problem. Only some directions can be provided. One way to construct a DFD is to start by identifying the major inputs and outputs.

In a DFD, data flows are identified by unique names. These names are chosen so that they convey some meaning about what the data is.

The ***data dictionary*** is a repository of various data flows defined in a DFD. The associated data dictionary states precisely the structure of each data flow in the DFD.

The data dictionary entry for weekly timesheet specifies that this data flow is composed of three basic
data entities—the employee name, employee ID, and many occurrences of the two-tuple consisting of regular hours and overtime hours. The last entity represents the daily working hours of the worker. The data dictionary also contains entries for specifying the different elements of a data flow.

```
weekly timesheet =
        Employee_name +
        Employee_Id +
        [Regular_hours + Overtime_hours] *

pay_rate =
        [Hourly | daily | weekly] +
        Dollar_amount

Employee_name =
        Last + First + Middle_initial

Employee_Id =
        digit + digit + digit + digit
```

Fig: Data dictionary

Once we have constructed a DFD and its associated data dictionary, we have to somehow verify that they are "correct." There can be no formal verification of a DFD, because what the DFD is modeling is not formally specified anywhere against which verification can be done. Human processes and rules of thumb must be used for verification. In addition to the walkthrough with the client, the analyst should look for common errors. Some common errors are

- Unlabeled data flows
- Missing data flows; information required by a process is not available
- Extraneous data flows; some information is not being used in the process
- Consistency not maintained during refinement
- Missing processes
- Contains some control information

Prototyping

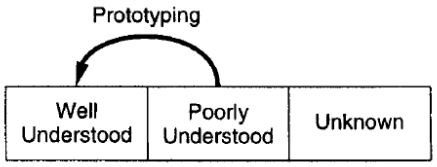
Prototyping takes a different approach to problem analysis as compared to modeling-based approaches. In prototyping, a partial system is constructed, which is then used by the client, users, and developers to gain a better understanding of the problem and the needs.

A software prototype can be defined as a partial implementation of a system whose purpose is to learn something about the problem being solved or the solution approach. Prototyping can also be

used to evaluate or check a design alternative (such a prototype is called a *design prototype).* Here we focus on prototyping used primarily for understanding the requirements.

There are two approaches to prototyping; throwaway and evolutionary.
In the *throwaway* approach the prototype is constructed with the idea that it will be discarded after the analysis is complete and the final system will be built from scratch. In the *evolutionary* approach, the prototype is built with the idea that it will eventually be converted into the final system.

The requirements of a system can be divided into three sets—those that are well understood those that are poorly understood, and those that are not known. In a throwaway prototype, the poorly understood requirements are the ones that should be incorporated. Based on the experience with the prototype, these requirements then become well understood, as shown in Figure



Throwaway prototyping

Requirements Specification

The final output is the software requirements specification document (SRS). The specification Activity might come after the entire analysis is complete. An analyst typically will analyze some parts of the problem and then write the requirements for that part. In practice, problem analysis and requirements specification activities overlap. Modeling generally focuses on the problem structure, not its external behavior. A good SRS needs to specify many things, some of which are not satisfactorily handled during modeling.
The modeling is essentially a tool to help obtain a thorough and complete knowledge about the proposed system. The SRS is written based on the knowledge acquired during analysis.
It is possible to develop the SRS without using formal modeling techniques. The basic aim of the structures used in modeling is to help in knowledge representation and problem partitioning.

Characteristics of an SRS

To properly satisfy the basic goals, an SRS should have certain properties and should contain different types of requirements.

A good SRS is:
1. Correct
2. Complete
3. Unambiguous
4. Verifiable
5. Consistent
6. Ranked for importance and/or stability
7. Modifiable

8. Traceable

An SRS is *correct* if every requirement included in the SRS represents something required in the final system.

An SRS is *complete* if everything the software is supposed to do and the responses of the software to all classes of input data are specified in the SRS.
Correctness ensures that which is specified is done correctly; completeness ensures that everything is indeed specified.

An SRS is *unambiguous* if and only if every requirement stated has one and only one interpretation. Requirements are often written in natural language, which are inherently ambiguous.

An SRS is *verifiable* if and only if every stated requirement is verifiable. A requirement is verifiable if there exists some cost-effective process that can check whether the final software meets that requirement.

An SRS is *consistent* if there is no requirement that conflicts with another. Terminology can cause inconsistencies; for example, different requirements may use different terms to refer to the same object. There may be logical or temporal conflict between requirements that causes inconsistencies.

An SRS is ranked for importance and/or stability if for each requirement the importance and the stability of the requirement are indicated. Stability of a requirement reflects the chances of it changing in future. It can be reflected in terms of the expected change volume.

An SRS is *modifiable* if its structure and style are such that any necessary change can be made easily while preserving completeness and consistency.

An SRS is traceable if the origin of each of its requirements is clear and if it facilitates the referencing of each requirement in future development. Forward traceability means that each requirement should be traceable to some design and code elements. Traceability aids verification and validation.

Components of an SRS

Completeness of specifications is difficult to achieve and even more difficult to verify. An SRS should specify all the requirements. Some of the system properties that an SRS should specify,
The basic issues an SRS must address are:

1. Functionality
2. Performance
3. Design constraints imposed on an implementation
4. External interfaces

Conceptually, any SRS should have these components. If the traditional approach to requirement analysis is being followed, then the SRS might even have portions corresponding to these. Functional requirements might be specified indirectly by specifying the services on the objects.

Functional requirements:

Functional requirements specify which outputs should be produced from the given inputs. They describe the relationship between the input and output of the system. For each functional requirement, a detailed description of all the data inputs and their source, the units of measure, and the range of valid inputs must be specified.

All the operations to be performed on the input data to obtain the output should be specified. This includes specifying the validity checks on the input and output data, parameters affected by the operation, and equations or other logical operations that must be used to transform the inputs into corresponding outputs.

An important part of the specification is the system behavior in abnormal situations, like invalid input (which can occur in many ways) or error during computation. The functional requirement must clearly state what the system should do if such situations occur.

Performance Requirements

This part of an SRS specifies the performance constraints on the software system. All the requirements relating to the performance characteristics of the system must be clearly specified. There are two types of performance requirements: static and dynamic.

Static requirements are those that do not impose constraint on the execution characteristics of the system. These include requirements like the number of terminals to be supported, the number of simultaneous users to be supported, and the number of files that the system has to process and their sizes.

Dynamic requirements specify constraints on the execution behavior of the system. These typically include response time and throughput constraints on the system. Response time is the expected time for the completion of an operation under specified circumstances. Throughput is the expected number of operations that can be performed in a unit time.

Design Constraints

There are a number of factors in the client's environment that may restrict the choices of a designer. Such factors include standards that must be followed, resource limits, operating environment, reliability and security requirements, and policies that may have an impact on the design of the system. An SRS should identify and specify all such constraints.

Standards Compliance: This specifies the requirements for the standards the system must follow. The standards may include the report format and accounting procedures.

Hardware Limitations: The software may have to operate on some existing or predetermined hardware, thus imposing restrictions on the design.

Reliability and Fault Tolerance: Fault tolerance requirements can place a major constraint on how the system is to be designed. Fault tolerance requirements often make the system more complex and expensive.

Security: Security requirements are particularly significant in defense systems and many database systems. Security requirements place restrictions on the use of certain commands, control access to data, provide different kinds of access requirements for different people, require the use of passwords and cryptography techniques, and maintain a log of activities in the system.

External Interface Requirements

All the interactions of the software with people, hardware, and other software should be clearly specified. For the user interface, the characteristics of each user interface of the software product should be specified.

A preliminary user manual should be created with all user commands, screen formats, an explanation of how the system will appear to the user, and feedback and error messages.

The interface requirement should specify the interface with other software the system will use or that will use the system. This includes the interface with the operating system and other applications.

Specification languages:

Requirements specification necessitates the use of some specification language. The language should support the desired qualities of the SRS— modifiability, understandability, unambiguous, and so forth. In addition, the language should be easy to learn and use. As one might expect, many of these characteristics conflict in the selection of a specification language. For example, to avoid ambiguity, it is best to use some formal language. But for ease of understanding a natural language might be preferable.

If formal languages are to be used, they are often used to specify particular properties or for specific parts of the system, and these formal specifications are generally contained in the overall SRS, which is in a natural language.

The major advantage of using a natural language is that both client and supplier understand the language. However, by the very nature of a natural language, it is imprecise and ambiguous. To reduce the drawbacks of natural languages, most often natural language is used in a structured fashion. In structured English (for example), requirements are broken into sections and paragraphs. Each paragraph is then broken into subparagraphs. Many organizations also specify strict uses of some words like "shall," "perhaps," and "should" and try to restrict the use of common phrases in order to improve the precision and reduce the verbosity and ambiguity.

Structure of Requirement document:

All the requirements for the system have to be included in a document that is clear and concise.

The IEEE standards recognize the fact that different projects may require their requirements to be organized differently, that is, there is no one method that is suitable for all projects. It provides different ways of structuring the SRS. The first two sections of the SRS are the same in all of them. The general structure of an SRS is given in Figure,

1. Introduction
    - 1.1 Purpose
    - 1.2 Scope
    - 1.3 Definitions, Acronyms,
    - 1.4 References
    - 1.5 Overview
2. Overall Description
    - 2.1 Product Perspective
    - 2.2 Product Functions
    - 2.3 User Characteristics
    - 2.4 General Constraints and Abbreviations

2.5 Assumptions and Dependencies
3. Specific Requirements
3.1 External Interface Requirements
3.1.1 User Interfaces
3.1.2 Hardware Interfaces
3.1.3 Software Interfaces
3.1.4 Communication Interfaces
3.2. Functional Requirements
3.2.1 Mode 1
3.2.1.1 Functional Requirement 1.1
3.2.1.n Functional Requirement l.n
3.2.m Mode m
3.2.m.l Functional Requirement m.l
3.2.m.n Functional Requirement m.n
3.3 Performance Requirements
3.4 Design Constraints
3.5 Attributes
3.6 Other Requirements

Fig: The general structure of and SRS

The introduction section contains the purpose, scope, overview, etc. of the requirements document. It also contains the references cited in the document and any definitions that are used.

Section 2 describes the general factors that affect the product and its requirements. A general overview is presented to make the understanding of the specific requirements easier.

Product perspective is essentially the relationship of the product to other products; defining if the product is independent or is a part of a larger product, and what the principal interfaces of the product are.

A general abstract description of the functions to be performed by the product is given. Schematic diagrams showing a general view of different functions and their relationships with each other can often be useful.

The specific requirements section (section 3 of the SRS) describes all the details that the software developer needs to know for designing and developing the system.

The external interface requirements section specifies all the interfaces of the software: to people, other software's, hardware, and other systems.

In the functional requirements section, the functional capacities of the system are described. For each functional requirement, the required inputs, desired outputs, and processing requirements will have to be specified.

The performance section should specify both static and dynamic performance requirements. All factors that constrain the system design are described in the performance constraints section.

The attributes section specifies some of the overall attributes that the system should have. Any requirement not covered under these is listed under other requirements.

Requirement validation:

The development of software starts with a requirements document, which is also used to determine eventually whether or not the delivered software system is acceptable. It is therefore important that the requirements specification contains no errors and specifies the client's requirements correctly.

The basic objective of the requirements validation activity is to ensure that the SRS reflects the actual requirements accurately and clearly. A related objective is to check that the SRS document is itself of "good quality"

Many different types of errors are possible, but the most common errors that occur can be classified in four types: omission, inconsistency, incorrect fact, and ambiguity.

Omission is a common error in requirements. In this type of error, some user requirement is simply not included in the SRS; the omitted requirement may be related to the behavior of the system, its performance, constraints, or any other factor. Omission directly affects the external completeness of the SRS.

Inconsistency can be due to contradictions within the requirements themselves or to incompatibility of the
stated requirements with the actual requirements of the client or with the environment in which the system will operate.

Incorrect fact The third common requirement error is ***incorrect fact*** Errors of this type occur when some fact recorded in the SRS is not correct.

Ambiguity. The fourth common error type is ***ambiguity.*** Errors of this type occur when there are some requirements that have multiple meanings, that is, their interpretation is not unique.
Some projects have collected data about requirement errors. On an average, a total of more than 250 errors were detected, and the percentage of different types of errors was:

Omission	Incorrect Fact	Inconsistency	Ambiguity
26%	10%	38%	26%

As requirements are generally textual documents that cannot be executed, inspections are eminently suitable for requirements validation.

Requirement Reviews:

Requirement review is a review by a group of people to find errors and point out other matters of concern in the requirements specifications of the system. The review group should include the author of the requirements document, someone who understands the need of the client, a person of the design team, and the persons responsible for maintaining the requirement document. It is also good practice to include people not directly involved with product development, like a software quality engineer.

One way to organize these meeting is to review items which are presented in checklists. Alternatively, the meeting can start with analyst explaining each item of SRS. The error detected in this review is greater than the sum of the error that would be detected.

Checklists are frequently used in reviews to focus the review effort and ensure that no major source of errors is overlooked by the reviewers. A good checklist depends on the project.
Some of the checklists are,

- Are all hardware resource defined
- Is each requirement is testable?
- Is initial state of the system is defined?
- Function required by client is specified?
- Possible future modification specified?

Other requirement review methods:

- Automated cross reference
- Reading
- Construct scenarios
- Prototyping

****

Function-oriented design

The design activity begins when the requirements document for the software to be developed is available and the architecture has been designed. During design we further refine the architecture.
Design focuses on the *module view.*
During design we determine what modules should the system have and which have to be developed. Sometimes, the module view may effectively be a module structure of each component in the architecture. That is, the design exercise determines the module structure of the components.

The design of a system is essentially a blueprint or a plan for a solution for the system. Here we consider a system to be a set of modules with clearly defined behavior which interact with each other in a defined manner to produce some behavior or services for its environment. A module of a system can be considered a system, with its own modules

Top level design and detailed design

The design process for software systems often has two levels. At the first level the focus is on deciding which modules are needed for the system, the specifications of these modules, and how the modules should be interconnected. This is called the ***system design or top-level design.***

In the second level, the internal design of the modules, or how the specifications of the module can be satisfied, is decided. This design level is often called ***detailed design or logic design.***
Detailed design essentially expands the system design to contain a more detailed description of the processing logic and data structures so that the design is sufficiently complete for coding.

A design methodology is a systematic approach to creating a design by applying of a set of techniques and guidelines. Most design methodologies focus on the system design, and do not reduce the design activity to a sequence of steps that can be blindly followed by the designer,

The function-oriented methods for design and describe one particular methodology—the structured design methodology— in some detail. In a function-oriented design approach, a system is viewed as a transformation function, transforming the inputs to the desired outputs.
The purpose of the design phase is to specify the components for this transformation function, so that each component is also a transformation function. That is, each module in design supports a functional abstraction. The basic output of the system design phase, when a function oriented design approach is being followed, is the definition of all the major data structures in the system, all the major modules of the system, and how the modules interact with each other.

Design Principles

The design of a system is *correct* if a system built precisely according to the design satisfies the requirements of that system. The goal is to find the *best* possible design within the limitations imposed by the requirements and the physical and social environment in which the system will operate.
A design should clearly be verifiable, ***complete*** (implements all the specifications), and ***traceable*** (all design elements can be traced to some requirements).
Efficiency of any system is concerned with the proper use of scarce resources by the system. If some resources are scarce and expensive, it is desirable that those resources be used efficiently. In computer systems, the resources that are most often considered for efficiency are processor time

and memory. An efficient system is one that consumes less processor time and requires less memory.

Simplicity is perhaps the most important quality criteria for software systems. We have seen that maintenance of software is usually quite expensive. Maintainability of software is one of the goals we have established. The design of a system is one of the most important factors affecting the maintainability of a system. A ***simple and understandable*** design will go a long way in making the job of the maintainer easier. Creating a simple design of a large system can be an extremely complex task that requires good engineering judgment.

Problem Partitioning and Hierarchy

For solving larger problems, the basic principle is the time-tested principle of "divide and conquer."

For software design, therefore, the goal is to divide the problem into manageably small pieces that can be solved separately. It is this restriction of being able to solve each part separately that makes dividing into pieces a complex task and that many methodologies for system design aim to address.

The basic rationale behind this strategy is the belief that if the pieces of a problem are solvable separately, the cost of solving the entire problem is more than the sum of the cost of solving all the pieces.

Abstraction

Abstraction is a very powerful concept that is used in all engineering disciplines. It is a tool that permits a designer to consider a component at an abstract level without worrying about the details of the implementation of the component.

An abstraction of a component describes the external behavior of that component without bothering with the internal details that produce the behavior. Abstraction is an indispensable part of the design process and is essential for problem partitioning. Partitioning essentially is the exercise in determining the components of a system. Abstraction is used for existing components as well as components that are being designed.

There are two common abstraction mechanisms for software systems: ***functional abstraction and data abstraction.***

In ***functional abstraction****,* a module is specified by the function it performs. Functional abstraction is the basis of partitioning in function-oriented approaches. The overall transformation function for the system is partitioned into smaller functions that comprise the system function. The decomposition of the system is in terms of functional modules.

In ***data abstraction,*** any entity in the real world provides some services to the environment to which it belongs. Data abstraction forms the basis for *object-oriented design.*

Modularity

If a system is partitioned into modules so that the modules are solvable and modifiable separately. It will be even better if the modules are also separately compliable. A system is considered *modular* if it consists of discreet components so that each component can be implemented separately, and a change to one component has minimal impact on other components.

Modularity is a clearly a desirable property in a system. Modularity helps in system debugging isolating the system problem to a component is easier if the system is modular; in system repair changing a part of the system is easy as it affects few other parts; and in system building a modular system can be easily built by "putting its modules together."

A software system cannot be made modular by simply chopping it into a set of modules. For modularity, each module needs to support a well defined abstraction and have a clear interface through which it can interact with other modules.

Modularity is where abstraction and partitioning come together. For easily understandable and maintainable systems, modularity is clearly the basic objective; partitioning and abstraction can be viewed as concepts that help achieve modularity.

Top-Down and Bottom-Up Strategies of Design:

A system consists of components, which have components of their own; indeed a system is a hierarchy of components.
The highest-level component corresponds to the total system. To design such a hierarchy there is two possible approaches: top-down and bottom-up.

The top-down approach starts from the highest-level component of the hierarchy and proceeds through to lower levels.
A ***top-down design*** approach starts by identifying the major components of the system, decomposing them into their lower-level components and iterating until the desired level of detail is achieved.
Top-down design method soften result in some form of ***stepwise refinement*** Starting from an abstract design, in each step the design is refined to a more concrete level, until we reach a level where no more refinement is needed and the design can be implemented directly.
A top-down approach is suitable only if the specifications of the system are clearly known and the system development is from scratch.

By contrast, a ***bottom-up*** approach starts with the lowest-level component of the hierarchy and proceeds through progressively higher levels to the top-level component.
A ***bottom-up*** design approach starts with designing the most basic or primitive components and proceeds to higher-level components that use these lower-level components. Bottom-up methods work with ***layers of abstraction***. Starting from the very bottom, operations that provide a layer of abstraction are implemented.
If a system is to be built from an existing system, a bottom-up approach is more suitable, as it starts from some existing components.

Module level concept:
A ***module*** is a logically separable part of a program. It is a program unit that is discreet and identifiable with respect to compiling and loading. In terms of common programming language constructs, a module can be a macro, a function, a procedure (or subroutine), a process, or a package. In systems using functional abstraction, a module is usually a procedure of function or a collection of these.
To produce modular designs, some criteria must be used to select modules so that the modules support well-defined abstractions and are solvable and modifiable separately. In a system using

functional abstraction, coupling and cohesion are two modularization criteria, which are often used together.

Coupling:

Two modules are considered independent if one can function completely without the presence of other. If two modules are independent, they are solvable and modifiable separately.
All the modules in a system cannot be independent of each other, as they must interact so that together they produce the desired external behavior of the system.
The more connections between modules, the more dependent they are in the sense that more knowledge about one module is required to understand or solve the other module.

Coupling between modules is the strength of interconnections between modules or a measure of interdependence among modules. In general, the more we must know about module A in order to understand module B, the more closely connected A is to B.

"Highly coupled" modules are joined by strong interconnections, while "loosely coupled" modules have weak interconnections. Independent modules have no interconnections.
Coupling increases with the complexity and obscurity of the interface between modules. To keep coupling low we would like to minimize the number of interfaces per module and the complexity of each interface.
The more complex each interface is, the higher will be the degree of coupling.

The type of information flow along the interfaces is the third major factor affecting coupling. There are two kinds of information that can flow along an interface: data or control. Passing or receiving control information means that the action of the module will depend on this control information, which makes it more difficult to understand the module and provide its abstraction.

Transfer of data information means that a module passes as input some data to another module and gets in return some data as output. This allows a module to be treated as a simple input-output function that performs some transformation on the input data to produce the output data.

	Interface Complexity	Type of Connection	Type of Communication
Low	Simple obvious	To module by name	Data
High	Complicated obscure	To internal elements	Control Hybrid

Table 6.1: Factors affecting coupling.

Cohesion

Cohesion is the concept that tries to capture this intra-module relationship. **Cohesion** of a module represents how tightly bound the internal elements of the module are to one another. Cohesion of a

module gives the designer an idea about whether the different elements of a module belong together in the same module.

The greater the cohesion of each module in the system, the lower the coupling between modules is. In general the Coincidental is the lowest level, and functional is the highest. These levels do not form a linear scale. Functional binding is much stronger than the rest, while the first two are considered much weaker than others. Cohesion of a module is considered the highest level of cohesion applicable to all elements in the module.

There are several levels of cohesion,

- Coincidental
- Logical
- Temporal
- Procedural
- Communicational
- Sequential
- Functional

Coincidental cohesion occurs when there is no meaningful relationship among the elements of a module.
Coincidental cohesion can occur if an existing program is "modularized" by chopping it into pieces and making different pieces modules.

A module has logical cohesion if there is some logical relationship between the elements of a module, and the elements perform functions that fall in the same logical class. A typical example of this kind of cohesion is a module that performs all the inputs or all the outputs.

Temporal cohesion is the same as logical cohesion, except that the elements are also related in time and are executed together. Modules that perform activities like "initialization," "clean-up," and "termination" are usually temporally bound.

A **procedurally cohesive** module contains elements that belong to a common procedural unit. For example, a loop or a sequence of decision statements in a module may be combined to form a separate module.

A module with ***communicational cohesion*** has elements that are related by a reference to the same input or output data. That is, in a communicational bound module, the elements are together because they operate on the same input or output data.

When the elements are together in a module because the output of one forms the input to another, we get ***sequential cohesion***. If we have a sequence of elements in which the output of one forms the input to another, sequential cohesion does not provide any guidelines on how to combine them into modules.

Functional cohesion is the strongest cohesion. In a functionally bound module, all the elements of the module are related to performing a single function. By function, we do not mean simply mathematical functions; modules accomplishing a single goal are also included.

Design Notation and Specification

In designing, a designer needs to record his views and decisions and to represent the design so that he can view it and play with it. For this, design notations are used.
Design notations are used during the process of design and are used to represent design or design decisions. They are for the designer so that they can quickly represent decisions in a compact manner that can evaluate and modify. These notations are frequently graphical.
Once the designer is satisfied with the design he has produced, the design is to be precisely specified in the form of a document.

Structure Charts

For a function-oriented design, the design can be represented graphically by structure charts. The structure of a program is made up of the modules of that program together with the interconnections between modules.
The structure chart of a program is a graphic representation of its structure. In a structure chart a module is represented by a box with the module name written in the box. An arrow from module A to module B represents that module A invokes module B. B is called the *subordinate* of A, and A is called the *super ordinate* of B. The arrow is labeled by the parameters received by B as input and the parameters returned by B as output, with the direction of flow of the input and output parameters represented by small arrows.
The parameters can be shown to be data (unfilled circle at the tail of the label) or control (filled circle at the tail).

As an example consider the structure of the following program, whose structure is shown in Figure,

```
main()
{
     int sum, n, N, a[MAX];
     readnums(a, &N); sort(a, N); scanf(&n);
     sum = add_n(a, n); printf(sum);
}

readnums(a, N)


int a[], *N;
{
     :
}

sort(a, N)
int a[], N;
{
     :
     if (a[i] > a[t]) switch(a[i], a[t]);
     :
}

/* Add the first n numbers of a */
add_n(a, n)
int a[], n;
{
     :
}
```

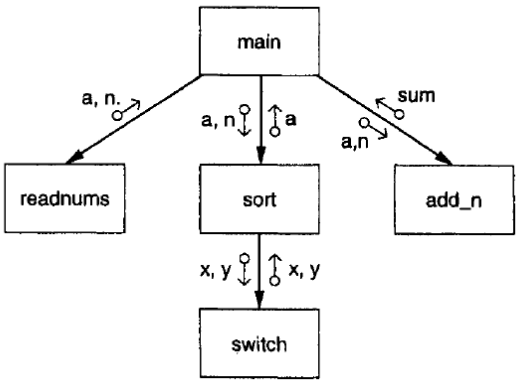


Fig: Structure-chart for sort program

In general, procedural information is not represented in a structure chart, and the focus is on representing the hierarchy of modules. However, there are situations where the designer may wish to communicate certain procedural information explicitly, like major loops and decisions. Such information can also be represented in a structure chart. For example, let us consider a situation where module A has subordinates B, C, and D, and A repeatedly calls the modules C and D. This can be represented by a looping arrow around the arrows joining the subordinates C and D to A, as shown in Figure

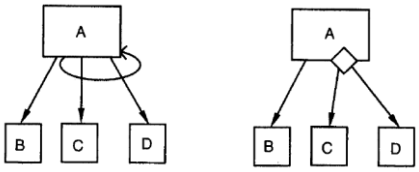


Fig: Iteration and decision representation.

All the subordinate modules activated within a common loop are enclosed in the same looping arrow.
Major decisions can be represented similarly. For example, if the invocation of modules C and D in module A depends on the outcome of some decision, that is represented by a small diamond in the box for A, with the arrows joining C and D coming out of this diamond as shown in above fig.

Modules in a system can be categorized into few classes.
There are some modules that obtain information from their subordinates and then pass it to their super ordinate. This kind of module is an *input module.* Similarly, there are *output modules that* take information from their super ordinate and pass it on to its subordinates. As the name suggests, the input and output modules are typically used for input and output of data from and to the environment. The input modules get the data from the sources and get it ready to be processed, and the output modules take the output produced and prepare it for proper presentation to the environment.
Then there are modules that exist solely for the sake of transforming data into some other form. Such a module is called a *transform module.* Most of the computational modules typically fall in this category.

Finally, there are modules whose primary concern is managing the flow of data to and from different subordinates. Such modules are called *coordinate modules.*
A module can perform functions of more than one type of module. A is a coordinate module and views its job as getting data X from one subordinate and passing it to another subordinate, which converts it to Y. Modules in actual systems are often composite modules.
The structure chart representation of the different types of modules is shown in Figure

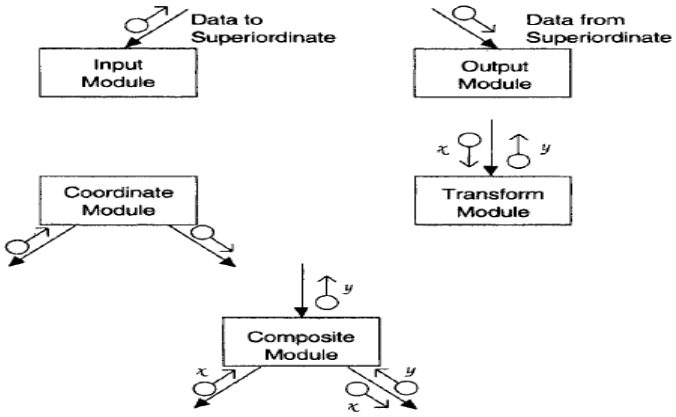


Fig: different types of modules.

Specification

The requirements document specifies the problem in the terminology of the problem domain. The problem specification is restated.
A design specification should define the major *data structures*, modules and their specifications, and design decisions.
During system design, the major data structures for the software are identified; without these, the system modules cannot be meaningfully defined during design. In the design specification, a formal definition of these *data structures* should be given.
Module specification is the major part of system design specification. All modules in the system should be identified when the system design is complete, and these modules should be specified in the document. During system design only the module specification is obtained, because the internal details of the modules are defined later.
The major *design decisions* made by the designers during the design process should be explained explicitly.
The choices that were available and the reasons for making a particular choice should be explained. This makes a design more visible and will help in understanding the design.

Structured Design Methodology (SDM)

Structured design methodology (SDM) views every software system as having some inputs that are converted into the desired outputs by the software system. The software is viewed as a transformation function that transforms the given inputs into the desired outputs, and the central problem of designing software systems is considered to be properly designing this transformation function.

The structured design methodology is primarily function-oriented and relies heavily on functional abstraction and functional decomposition.

The concept of the structure of a program has at the heart of the structured design method. During design, structured design methodology aims to control and influence the structure of the final program. The aim is to design a system so that programs implementing the design would have a hierarchical structure, with functionally cohesive modules and as few interconnections between modules as possible.

Factoring is the process of decomposing a module so that the bulk of its work is done by its subordinates. A system is said to be completely factored if all the actual processing is accomplished by bottom-level atomic modules and if non-atomic modules largely perform the jobs of control and coordination.

The overall strategy is to identify the input and output streams and the primary transformations that have to be performed to produce the output. High-level modules are then created to perform these major activities, which are later refined.
There are four major steps in SDM strategy,
1. Restate the problem as a data flow diagram
2. Identify the input and output data elements
3. First-level factoring
4. Factoring of input, output, and transform branches

Restate the Problem as a Data Flow Diagram:
To use the SD methodology, the first step is to construct the data flow diagram for the problem. The DFD during design represents how the data will flow in the system when it is built. In this modeling, the major transforms or functions in the software are decided, and the DFD shows the major transforms that the software will have and how the data will flow through different transforms.
Drawing a DFD for design is a very creative activity in which the designer visualizes the eventual system and its processes and data flows.
Consider the problem of determining the number of different words in an input file. The data flow diagram for this problem is shown in Figure,

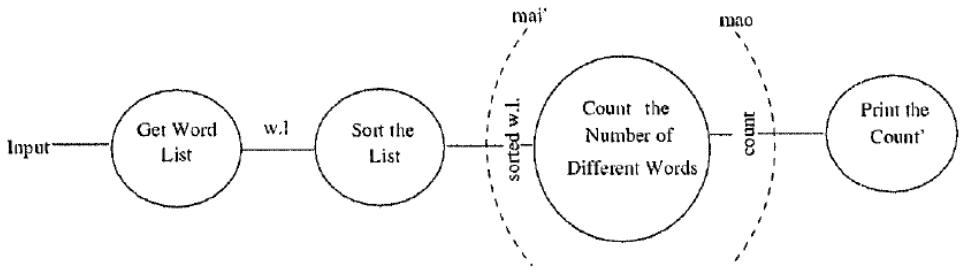


Fig: DFD for world count problem

This problem has only one input data stream, the input file, while the desired output is the count of different words in the file.

To transform the input to the desired output, the first thing we do is form a list of all the words in the file. It is best to then sort the list, as this will make identifying different words easier. This sorted fist is then used to count the number of different words, and the output of this transform is the desired count, which is then printed. This sequence of data transformation is what we have in the data flow diagram.

Identify the Most Abstract Input and Output Data Elements

Once the data flow diagram is ready, the next step is to identify the highest abstract level of input and output. *The most abstract input data elements* are those data elements in the data flow diagram that are farthest removed from the physical inputs but can still be considered inputs to the system. The most abstract input data elements often have little resemblance to the actual physical data.

Most abstract input (MAI) data elements are recognized by starting from the physical inputs and traveling toward the outputs in the data flow diagram, until the data elements are reached that can no longer be considered incoming. The aim is to go as far as possible from the physical inputs, without losing the incoming nature of the data element. This process is performed for each input stream.

Similarly, we identify the *most abstract output data elements (MAO)* by starting from the outputs in the data flow diagram and traveling toward the inputs. These are the data elements that are most removed from the actual outputs but can still be considered outgoing.
The MAO data elements may also be considered the logical output data items, and the transforms in the data flow diagram after these data items are basically to convert the logical output into a form in which the system is required to produce the output.

There will usually be some transforms left between the most abstract input and output data items. These *central transforms* perform the basic transformation for the system, taking the most abstract input and transforming it into the most abstract output. Consider this *central transform for factoring the system.*
For different word count problem the sorted word list is the MAI and count is the MAO. Thus we have one central transform, count-number-of-different-words, which has one input and one output data item.

First-Level Factoring

After identifying the central transforms and the most abstract input and output data items, we are ready to identify some modules for the system. We first specify a main module, whose purpose is to invoke the subordinates. The main module is therefore a coordinate module.
For each of the most abstract input data items, an immediate subordinate module to the main module is specified. Each of these modules is an input module, whose purpose is to deliver to the main module the most abstract data item for which it is created.

Similarly, for each most abstract output data item, a subordinate module that is an output module that accepts data from the main module is specified. Each of the arrows connecting these input and output subordinate modules is labeled with the respective abstract data item flowing in the proper direction.

Finally, for each central transform, a module subordinate to the main one is specified. These modules will be transform modules, whose purpose is to accept data from the main module, and then return the appropriate data back to the main module.

The data items coming to a transform module from the main module are on the incoming arcs of the corresponding transform in the data flow diagram. The data items returned are on the outgoing arcs of that transform. Note that here a module is created for a transform, while input/output modules are created for data items. The structure after the first-level factoring of the word-counting problem is shown in Figure,

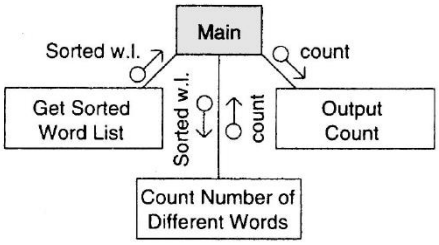


fig: first level factoring

Factoring the Input, Output, and Transform Branches:

Each of the input, output, and transformation modules must be considered for factoring. To factor an input module, the transform in the data flow diagram that produced the data item is now treated as a central transform. The process performed for the first-level factoring is repeated here with this new central transform, with the input module being considered the main module. A subordinate input module is created for each input data stream coming into this new central transform, and a subordinate transform module is created for the new central transform. The new input modules now created can then be factored again, until the physical inputs are reached.

The factoring of the output modules is symmetrical to the factoring of the input modules. For an output module we look at the next transform to be applied to the output to bring it closer to the ultimate desired output. This now becomes the central transform, and an output module is created for each data stream going out of this transform.

One way to factor a transform module is to treat it as a problem in its own right and start with a data flow diagram for it. The inputs to the data flow diagram are the data coming into the module and the outputs are the data being returned by the module. Each transform in this data flow diagram represents a sub transform of this transform. The central transform can be factored by creating a subordinate transform module for each of the transforms in data flow diagram. The factoring of the input module get-sorted-list in the first-level structure is shown in Figure

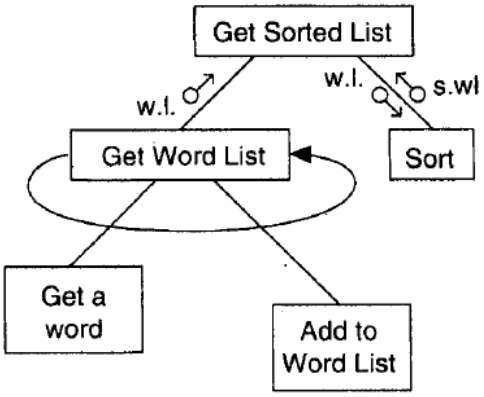


Fig: factoring the input module

Design Heuristics

The design steps mentioned earlier do not reduce the design process to a series of steps that can be followed blindly.

A module should be split into separate modules only if the cohesion of the original module was low, the resulting modules have a higher degree of cohesion, and the coupling between modules does not increase.

Fan-in of a module is the number of arrows coming in the module, indicating the number of super ordinates of a module. Fan-out of a module is the number of arrows going out of that module, indicating the number of subordinates of the module.

Transaction Analysis:

The structured design technique discussed earlier is called transform analysis, where most of the transforms in the data flow diagram have a few inputs and a few outputs. In such situations the transform analysis can be supplemented by transaction analysis, and the detailed data flow diagram of the transform splitting the input may look like the DFD shown in Figure 6.10.

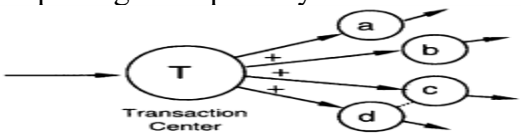
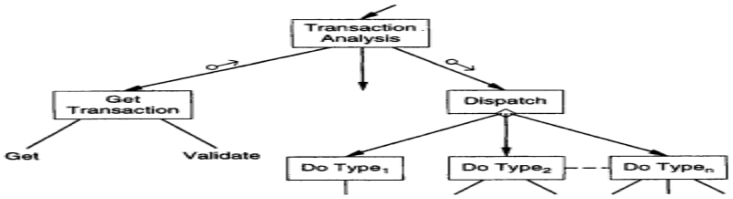


Figure 6.10: DFD for transaction analysis.



The module splitting the input is called the transaction center; it need not be a central transform and may occur on either the input branch or the output branch of the data flow diagram of the system. One of the standard ways to convert a data flow diagram of the form shown in Figure 6.10 into a structure chart is to have an input module that gets the analyzed transaction and a dispatch

module that invokes the modules for the different transactions. For smaller systems the analysis and the dispatching can be done in the transaction center module itself, giving rise to a flatter structure.
For designing systems that require transaction analysis, start with a data flow diagram, as in transform analysis, and identify the transform centers. Factor the data flow diagram, as is done in transform analysis. For the modules corresponding to the transform centers, draw the detailed data flow diagram.

Verification
The output of the system design phase, like the output of other phases in the development process, should be verified before proceeding with the activities of the next phase. If the design is expressed in some formal notation for which analysis tools are available, then through tools it can be checked for internal consistency.
The most common approach for verification is design review or inspections.

Design Reviews:
The purpose of design reviews is to ensure that the design satisfies the requirements and is of "good quality." If errors are made during the design process, they will ultimately reflect themselves in the code and the final system.

The system design review process is similar to the inspection process, in that a group of people get together to discuss the design with the aim of revealing designs errors or undesirable properties.
The review group must include a member of both the system design team and the detailed design team, the author of the requirements document, the author responsible for maintaining the design document, and an independent software quality engineer.

During reviews, elements of design that are not conducive to modification and expansion or elements that fail to conform to design standards should also be considered "errors."
A Sample Checklist: The use of checklists can be extremely useful for any review. The checklist can be used by each member during private study of the design and during the review meeting. For best results the checklist should be tailored to the project at hand, to uncover problem-specific errors.
General items that can be used to construct a checklist for a design review,

- Is each of the functional requirements taken into account?
- Are there analyses to demonstrate that performance requirements can be met?
- Are all assumptions explicitly stated, and are they acceptable?
- Are there any limitations or constraints on the design beyond those in the requirements?
- Are external specifications of each module completely specified?
- Are all the data formats consistent with the requirements?
- Are the operator and user interfaces properly addressed?
- Is the design modular, and does it conform to local standards?

****